AI-Assisted Enterprise Code Enhancement Report

Project: ERP.Api — Vessel management API

Source: ERP.Api/Controllers/VesselsController.cs · Tool: Cursor AI · May 18, 2026

1. Existing Problems in the Original Code

- Validation logic lives in the HTTP controller (transport + domain mixed).
- Email validation duplicated across Create and Update actions.
- No IMO format guard (7-digit maritime convention).
- No structured logging on persistence failures.
- Entity mapping and SaveChanges tightly coupled in the action.

2. Original Code Snippet

```
[HttpPost]
public async Task<ActionResult<VesselListItem>> Create(
    [FromBody] CreateVesselRequest request, CancellationToken ct)
{
    if (string.IsNullOrWhiteSpace(request.Name) || ...)
        return BadRequest(...);

    var name = request.Name.Trim();
    var imo = request.Imo.Trim();
    var email = request.Email.Trim();

    if (!EmailValidator.IsValid(email))
        return BadRequest(...);

    if (await _db.Vessels.AnyAsync(v => v.Imo == imo, ct))
        return Conflict(...);

    var entity = new Vessel { Name = name, Imo = imo, ... };
    _db.Vessels.Add(entity);
    await _db.SaveChangesAsync(ct);
    return CreatedAtAction(...);
}
```

3. AI Prompt Used

```
Enhance this enterprise C# ASP.NET Core controller action using modern .NET
clean architecture principles. Focus: structured logging, reusable validation,
guard clauses, null safety, SOLID, separation of concerns, helper extraction,
reduced cognitive complexity, production-ready standards.
```

4. AI-Enhanced Version (excerpt)

```
[HttpPost]
public async Task<ActionResult<VesselListItem>> Create(...)
{
    if (request is null)
        return BadRequest(...);

    if (!TryNormalizeCreateRequest(request, out var normalized, out var err))
        return BadRequest(new { message = err });

    try {
        if (await _db.Vessels.AnyAsync(v => v.Imo == normalized.Imo, ct))
            return Conflict(...);
        var entity = MapNewVessel(normalized);
        _db.Vessels.Add(entity);
        await _db.SaveChangesAsync(ct);
```

```
        return CreatedAtAction(nameof(GetAll), null, ToListItem(entity));
    }
    catch (DbUpdateException ex) {
        _logger.LogError(ex, "Database error in {Controller}.{Action}",
            nameof(VesselsController), nameof(Create));
        return StatusCode(500, ...);
    }
}

// Helpers: TryNormalizeCreateRequest, IsValidImo, MapNewVessel, ToListItem
```

5. Expert-Level Improvements

- Guard clauses — early returns before database access.
- Structured logging — ILogger with controller, action, and IMO context.
- Reusable validation — shared TryNormalizeCreateRequest / IsValidImo.
- Separation of concerns — mapping in dedicated helpers.
- SOLID (SRP) — controller orchestrates HTTP only.
- Production safety — DbUpdateException with safe client message.

6. Productivity Observation

- AI-assisted development significantly reduced refactoring effort for repetitive validation, response handling, and logging boilerplate across Create and Update.

- Cursor AI accelerated clean-code transformation by suggesting reusable helpers, structured logging patterns, and maintainable refactoring in minutes instead of hours.